Online Resource 1

PLENA implementation details and reproducible computational workflow

Article: Scalable Stochastic Characterization of Urban Drainage Networks: Applying PLENA to 239 Seoul Catchments

Journal	Stochastic Environmental Research and Risk Assessment
Authors	Changmin Park, Minsoo Seok, Yongwon Seo
Affiliation	Department of Civil Engineering, Yeungnam University, Gyeongsan, Republic of Korea
Corresponding author	Yongwon Seo (yongwon.seo@yu.ac.kr)
Repository	https://github.com/seokminsu99-byte/FMT_plug_PLENA

Resource caption

PLENA implementation details and reproducible computational workflow. This PDF contains supplementary figures, descriptions of the input-file structure and flow-direction coding, compilation and execution procedures, output-file descriptions, implementation notes, and additional computational-performance results.

1. Scope

PLENA (Program for Large-scale Evaluation of Network Analysis) is the C++17 implementation used in the accompanying study for stochastic Gibbs-model drainage-network generation, width-function calculation, Nash-Sutcliffe efficiency (NSE) comparison, and representative beta-class estimation. This resource documents the checked source implementation and the workflow needed to interpret the supplementary figures and output files.

The source code associated with this resource is available at https://github.com/seokminsu99-byte/FMT_plug_PLENA. The repository contains the checked C++ source, example input, direction-code explanation, license, citation metadata, and the two SERRA online-resource files.

2. Reproducible computational workflow

2.1 Build requirements

A compiler supporting C++17 is required. The repository build command is:

```
g++ -O2 -std=c++17 src/main_EN_ver.cpp -o PLENA
```

2.2 Input-file structure

The plain-text input contains: (1) the number of rows and columns, (2) the one-based outlet row and column, and (3) the drainage-direction matrix. Direction codes are 0 = inactive/no pipe, 1 = east, 2 = south, 3 = west, and 4 = north. The outlet must lie within the active-cell mask.

2.3 Execution

Run PLENA with an input path (for example, `PLENA example.txt`) or start it without an argument and select a listed text file. The user enters the exponent k in beta = 10^k. Width-function analysis is optional. When batch width functions are requested, the user selects the candidate exponent range and replicate count; a thread cap of 0 requests automatic selection from `hardware_concurrency()`.

2.4 Output files

The primary result text file contains the generated direction matrix, travel-time matrix, flow-accumulation matrix, and q(t). Optional width-function analysis writes FD and LS matrices, a width-functions CSV file, and an NSE text report. The NSE report provides candidate-class summaries used to compare simulated mean width functions with the observed width function.

3. Checked implementation details

3.1 Contiguous matrix storage

The checked Matrix type stores cells in a single row-major integer vector. This layout improves spatial locality during repeated neighbor access in candidate updates, reachability checks, travel-time calculations, and width-function construction. Figure S1 illustrates the cache-locality rationale.

3.2 Candidate screening and outlet reachability

Each proposed direction change is screened for a repeated direction, movement outside the grid, movement to an inactive cell, and immediate opposite-direction conflict with a neighboring cell. A breadth-first search from the outlet then checks whether every active cell remains outlet-reachable; infeasible candidates are rejected.

3.3 Scale-proportional stochastic updates

The implementation uses a size-normalized update budget $I = \text{alpha}\ nm$, where n and m are the grid dimensions. The checked study configuration implements $I = 10nm$. This prevents the expected update opportunity per grid cell from decreasing solely because a catchment grid is larger.

3.4 Parallel batch execution

Candidate beta classes and stochastic realizations are independent tasks. PLENA uses a C++ worker pool with atomic task indexing and an optional user thread cap. Figure S2 documents automatic 24-thread execution and high processor utilization in the captured run.

3.5 Computational-performance comparison

Figure S3 compares generation times for the conventional MATLAB implementation and PLENA over the benchmark network sizes reported in the study. The logarithmic time axis shows the increasing runtime separation as network size grows. The figure supports the manuscript's reported acceleration range under the tested hardware and configurations; it is not a hardware-independent guarantee.

3.6 Interpretation boundary

PLENA and the beta class describe drainage-network structure through outlet-referenced flow-distance patterns. They do not by themselves represent rainfall forcing, pipe capacity, pumps, downstream water levels, surface storage, inundation depth, or flood probability.

Figure S1. Contiguous storage and cache locality

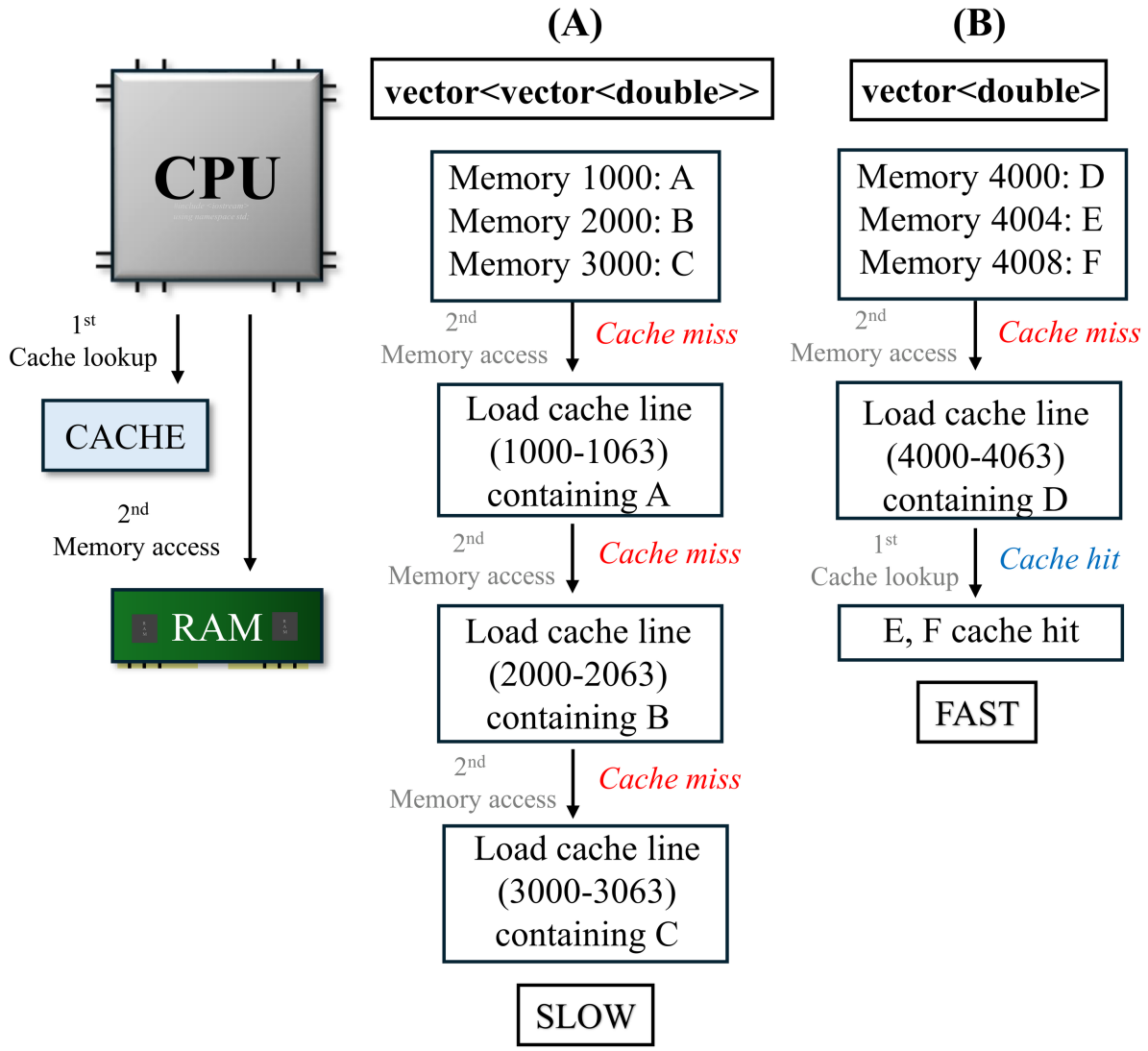

Figure S1. Schematic comparison of nested and contiguous memory layouts. Separate nested containers can require repeated cache-line loads during neighboring-cell access, whereas row-major contiguous storage places adjacent cell values together and improves cache locality. This figure explains the Matrix storage strategy used in PLENA.

Figure S2. Parallel execution

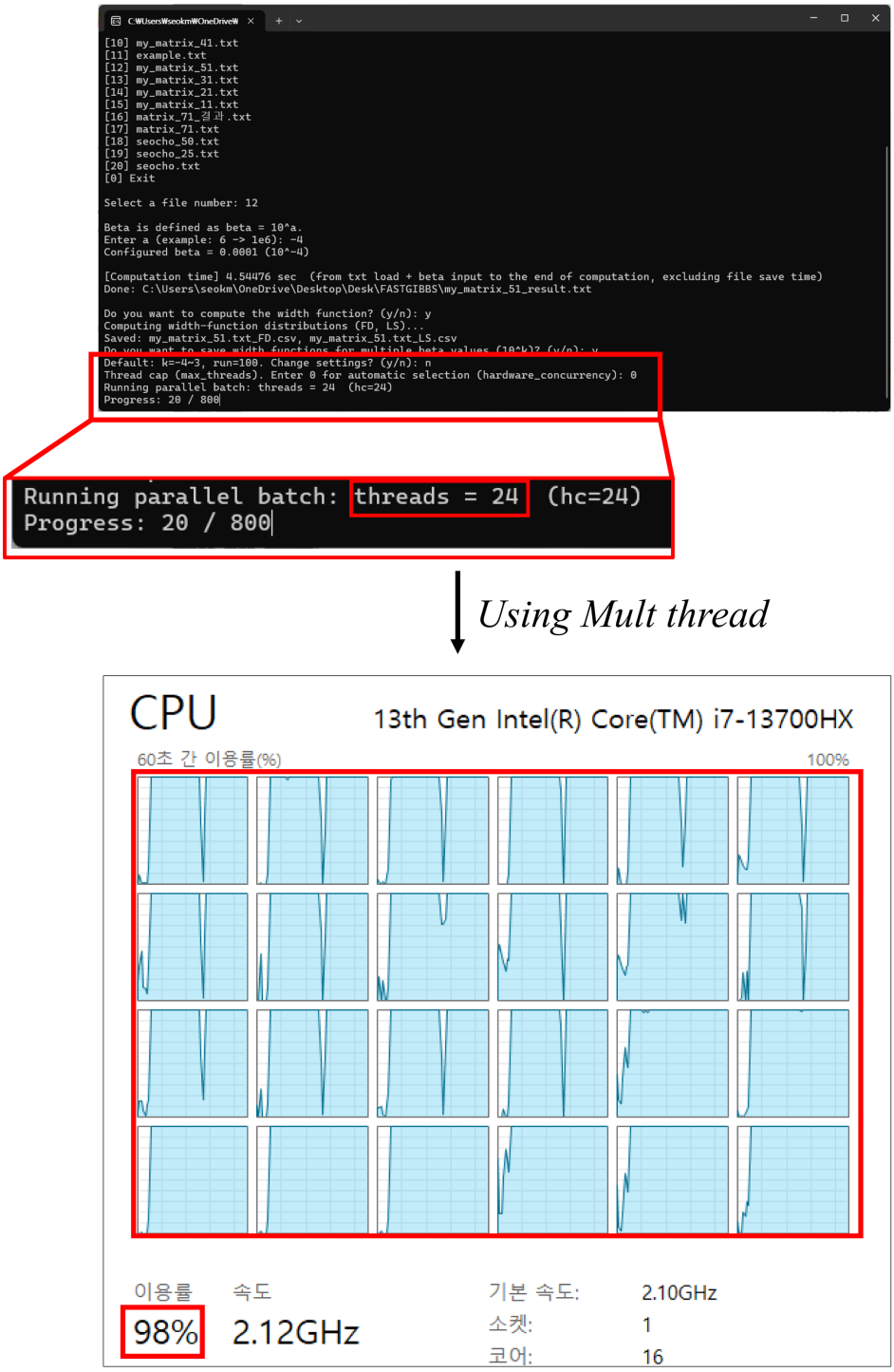

Figure S2. PLENA console output and processor utilization during parallel width-function estimation. The captured run reports automatic use of 24 worker threads, while the operating-system performance panel shows high processor utilization. The screenshot documents use of the implemented parallel batch workflow; utilization may vary with hardware and workload.

Figure S3. Computational-performance comparison

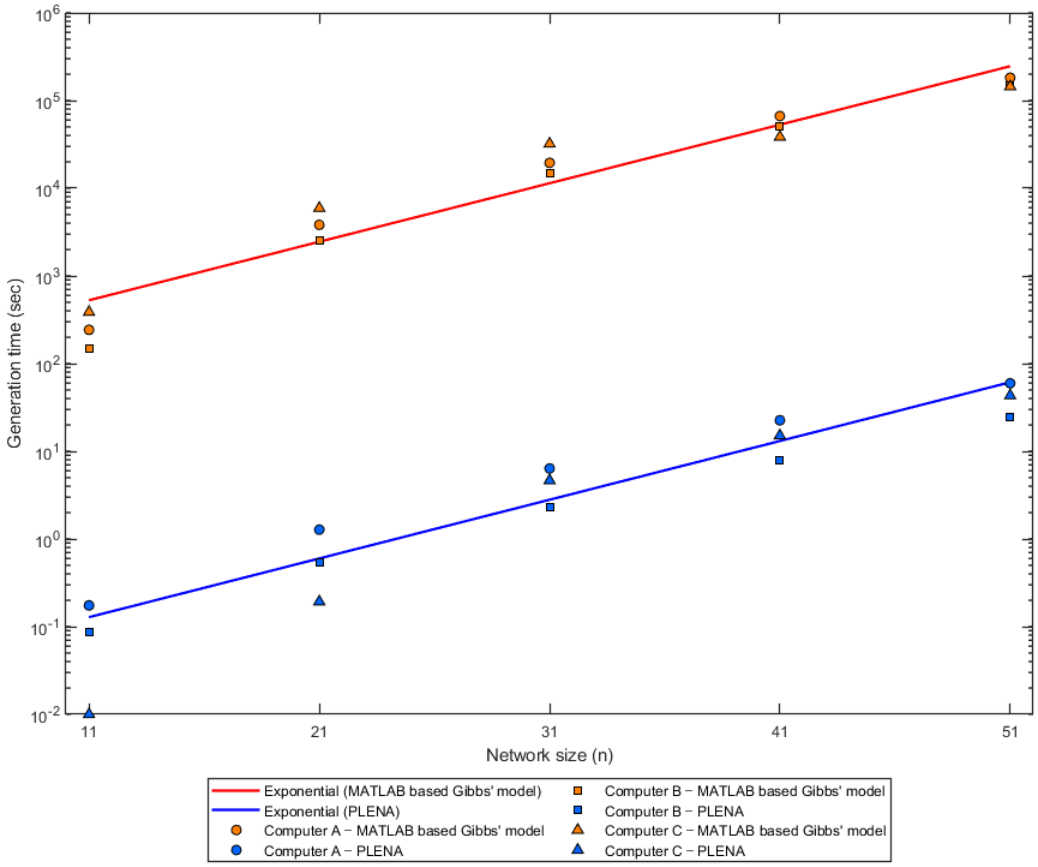

Figure S3. Generation-time comparison between the MATLAB-based Gibbs-model implementation and PLENA across the tested network sizes. The y-axis is logarithmic. PLENA remains below the MATLAB implementation across the benchmark sizes, and the separation grows with network size. Values represent the tested configurations and computing environment reported in the study.